

FIAP - MBA em Data Engineering

Data Engineering Programming

Trabalho Final – Projeto Pyspark

Aldrin Lucas Bethiol – RM371727

João Vitor Pereira Pinto – RM370585

Maria Eduarda Magalhães Moura – RM372127

Pedro Sanders Garcia – RM372503

Professor: Marcelo Barbosa Pinto

Repositório público no GitHub:
github.com/Abethiol/fiap-pyspark

1. Escopo de negócio atendido

O projeto implementa um pipeline Pyspark que gera um relatório de pedidos de venda cujos pagamentos foram recusados (`status=false`) e classificados como legítimos na avaliação de fraude (`avaliacao_fraude.fraude=false`), restrito a pedidos de 2025, contendo: identificador do pedido, UF, forma de pagamento, valor total do pedido e data do pedido. O relatório é ordenado por UF, forma de pagamento e data de criação, e gravado em formato Parquet.

2. Critérios específicos de avaliação atendidos

#	Critério	Onde foi implementado
1	Schemas explícitos	<code>src/config/pedidos_schema.py</code> , <code>src/config/pagamentos_schema.py</code> aplicados em <code>DataReader</code>
2	Orientação a objetos	Todos os componentes encapsulados em classes
3	Injeção de dependências	<code>src/main.py</code> (Aggregation Root)
4	Configurações centralizadas	<code>src/config/app_config.py</code>
5	Sessão Spark	<code>src/session/spark_session_manager.py</code>
6	Leitura e escrita de dados	<code>src/io/data_reader.py</code> , <code>src/io/data_writer.py</code>
7	Lógica de negócio	<code>src/business/business_logic.py</code>
8	Orquestração do pipeline	<code>src/pipeline/pipeline.py</code>
9	Logging	Configurado e utilizado em <code>BusinessLogic</code>
10	Tratamento de erros	<code>try/except</code> em <code>BusinessLogic.process</code> e <code>Pipeline.run</code>
11	Empacotamento	<code>pyproject.toml</code> , <code>requirements.txt</code> , <code>README.md</code> , <code>MANIFEST.in</code>
12	Testes unitários	<code>tests/test_business_logic.py</code> (pytest)

3. Evidências dos códigos-fonte

src/main.py (Aggregation Root) - linhas 1-20

```
1 from src.config.app_config import AppConfig
2 from src.session.spark_session_manager import SparkSessionManager
3 from src.io.data_reader import DataReader
4 from src.io.data_writer import DataWriter
5 from src.business.business_logic import BusinessLogic
6 from src.pipeline.pipeline import Pipeline
7
8
9 def main():
10     config = AppConfig()
11     spark = SparkSessionManager(config).get_spark()
12
13     try:
14         reader = DataReader(spark, config)
15         writer = DataWriter(config)
16         business_logic = BusinessLogic()
17
18         pipeline = Pipeline(
19             reader,
20             business_logic,
```

src/main.py - linhas 21-31

```
21         writer
22     )
23
24     pipeline.run()
25
26     finally:
27         spark.stop()
28
29
30 if __name__ == "__main__":
31     main()
```

src/config/app_config.py - linhas 1-20

```
1 import yaml
2
3
4 class AppConfig:
5
6     def __init__(self, config_file="src/config/settings.yaml"):
7         with open(config_file, "r", encoding="utf-8") as file:
8             self.config = yaml.safe_load(file)
9
10    @property
11    def pedidos_path(self):
12        return self.config["paths"]["pedidos"]
13
14    @property
15    def pagamentos_path(self):
16        return self.config["paths"]["pagamentos"]
17
18    @property
19    def output_path(self):
20        return self.config["paths"]["output"]
```

src/config/app_config.py - linhas 21-24

```
21
22    @property
23    def app_name(self):
24        return self.config["spark"]["app_name"]
```

src/config/settings.yaml - linhas 1-12

```
1 spark:
2   app_name: "FIAP PySpark"
3
4 paths:
5   pedidos: "data/datasets-csv-pedidos/data/pedidos"
6   pagamentos: "data/dataset-json-pagamentos/data/pagamentos"
7   output: "output/relatorio"
8
9 file_options:
10  pedidos_csv:
11    header: true
12    sep: ";"
```

src/config/pedidos_schema.py - linhas 1-17

```

1 from pyspark.sql.types import (
2     StructType,
3     StructField,
4     StringType,
5     DoubleType,
6     LongType
7 )
8
9 PEDIDOS_SCHEMA = StructType([
10     StructField("ID_PEDIDO", StringType(), True),
11     StructField("PRODUTO", StringType(), True),
12     StructField("VALOR_UNITARIO", DoubleType(), True),
13     StructField("QUANTIDADE", LongType(), True),
14     StructField("DATA_CRIACAO", StringType(), True),
15     StructField("UF", StringType(), True),
16     StructField("ID_CLIENTE", StringType(), True)
17 ])

```

src/config/pagamentos_schema.py - linhas 1-20

```

1 from pyspark.sql.types import (
2     StructType,
3     StructField,
4     StringType,
5     DoubleType,
6     BooleanType
7 )
8
9 PAGAMENTOS_SCHEMA = StructType([
10     StructField(
11         "avaliacao_fraude",
12         StructType([
13             StructField("fraude", BooleanType(), True),
14             StructField("score", DoubleType(), True)
15         ]),
16         True
17     ),
18     StructField("data_processamento", StringType(), True),
19     StructField("forma_pagamento", StringType(), True),
20     StructField("id_pedido", StringType(), True),

```

src/config/pagamentos_schema.py - linhas 21-23

```

21     StructField("status", BooleanType(), True),
22     StructField("valor_pagamento", DoubleType(), True)
23 ])

```

src/session/spark_session_manager.py - linhas 1-13

```
1 from pyspark.sql import SparkSession
2
3 class SparkSessionManager:
4
5     def __init__(self, config):
6         self.config = config
7
8     def get_spark(self):
9         return (
10             SparkSession.builder
11                 .appName(self.config.app_name)
12                 .getOrCreate()
13         )
```

src/io/data_reader.py - linhas 1-20

```
1 from src.config.pedidos_schema import PEDIDOS_SCHEMA
2 from src.config.pagamentos_schema import PAGAMENTOS_SCHEMA
3
4
5 class DataReader:
6
7     def __init__(self, spark, config):
8         self.spark = spark
9         self.config = config
10
11     def read_pedidos(self):
12         return (
13             self.spark.read
14                 .option("header", True)
15                 .option("delimiter", ";")
16                 .schema(PEDIDOS_SCHEMA)
17                 .csv(self.config.pedidos_path)
18         )
19
20     def read_pagamentos(self):
```

src/io/data_reader.py - linhas 21-25

```
21         return (
22             self.spark.read
23                 .schema(PAGAMENTOS_SCHEMA)
24                 .json(self.config.pagamentos_path)
25         )
```

src/io/data_writer.py - linhas 1-14

```

1 from pyspark.sql import DataFrame
2
3 class DataWriter:
4     def __init__(self, config):
5         self.config = config
6
7     def write_parquet(self, df: DataFrame):
8         (
9             df
10            .orderBy("UF", "FORMA_PAGAMENTO", "DATA_CRIACAO")
11            .write
12            .mode("overwrite")
13            .parquet(self.config.output_path)
14        )

```

src/business/business_logic.py - linhas 1-20

```

1 import logging
2 from pyspark.sql import DataFrame
3 from pyspark.sql.functions import col, year, to_timestamp
4
5 class BusinessLogic:
6     def __init__(self):
7         logging.basicConfig(
8             level=logging.INFO,
9             format="%(asctime)s - %(levelname)s - %(message)s"
10        )
11
12    def process(self,
13                pedidos_df: DataFrame,
14                pagamentos_df: DataFrame,
15                ) -> DataFrame:
16
17        try:
18            logging.info("Iniciando processamento.")
19
20            pedidos_2025 = pedidos_df.filter(

```

src/business/business_logic.py - linhas 21-40

```

21         year(
22             to_timestamp(
23                 col("DATA_CRIACAO"),
24                 "yyyy-MM-dd'T'HH:mm:ss"
25             )
26         ) == 2025
27     )
28
29     logging.info("Pedidos de 2025 filtrados.")
30
31     pagamentos_validos = pagamentos_df.filter(
32         (col("status") == False) &
33         (col("avaliacao_fraude.fraude") == False)
34     )
35
36     logging.info("Pagamentos válidos filtrados.")
37
38     resultado = (
39         pedidos_2025.alias("p")
40         .join(

```

src/business/business_logic.py - linhas 41-60

```

41         pagamentos_validos.alias("pg"),
42         col("p.ID_PEDIDO") == col("pg.id_pedido"),
43         "inner"
44     )
45     .select(
46         col("p.ID_PEDIDO"),
47         col("p.UF"),
48         col("pg.forma_pagamento").alias("FORMA_PAGAMENTO"),
49         (
50             col("p.VALOR_UNITARIO") *
51             col("p.QUANTIDADE")
52         ).alias("VALOR_TOTAL_PEDIDO"),
53         col("p.DATA_CRIACAO")
54     )
55 )
56
57 logging.info("Processamento concluído.")
58
59 return resultado
60

```

src/business/business_logic.py - linhas 61-63

```

61     except Exception as e:
62         logging.error(f"Erro durante o processamento: {e}")
63         raise

```


src/pipeline/pipeline.py - linhas 1-20

```
1 import logging
2
3 class Pipeline:
4     def __init__(self, reader, business_logic, writer):
5         self.reader = reader
6         self.business_logic = business_logic
7         self.writer = writer
8
9     def run(self):
10        try:
11            logging.info("Iniciando pipeline.")
12
13            df_pedidos = self.reader.read_pedidos()
14            df_pagamentos = self.reader.read_pagamentos()
15
16            resultado = self.business_logic.process(
17                df_pedidos,
18                df_pagamentos
19            )
20
```

src/pipeline/pipeline.py - linhas 21-27

```
21            self.writer.write_parquet(resultado)
22
23            logging.info("Pipeline finalizado.")
24
25        except Exception as e:
26            logging.error(f"Erro no pipeline: {e}")
27            raise
```

tests/test_business_logic.py - linhas 1-20

```
1 import pytest
2 from pyspark.sql import SparkSession
3 from pyspark.sql.types import (
4     StructType,
5     StructField,
6     StringType,
7     DoubleType,
8     LongType,
9     BooleanType
10 )
11
12 from src.business.business_logic import BusinessLogic
13
14
15 @pytest.fixture(scope="session")
16 def spark():
17     spark = (
18         SparkSession.builder
19         .master("local")
20         .appName("test")
```

tests/test_business_logic.py - linhas 21-40

```
21         .getOrCreate()
22     )
23
24     yield spark
25
26     spark.stop()
27
28
29 def test_process_filters_correctly(spark):
30     pedidos_schema = StructType([
31         StructField("ID_PEDIDO", StringType(), True),
32         StructField("PRODUTO", StringType(), True),
33         StructField("VALOR_UNITARIO", DoubleType(), True),
34         StructField("QUANTIDADE", LongType(), True),
35         StructField("DATA_CRIACAO", StringType(), True),
36         StructField("UF", StringType(), True),
37         StructField("ID_CLIENTE", StringType(), True)
38     ])
39
40     pagamentos_schema = StructType([
```

tests/test_business_logic.py - linhas 41-60

```

41         StructField("id_pedido", StringType(), True),
42         StructField("forma_pagamento", StringType(), True),
43         StructField("valor_pagamento", DoubleType(), True),
44         StructField("status", BooleanType(), True),
45         StructField(
46             "avaliacao_fraude",
47             StructType([
48                 StructField("fraude", BooleanType(), True),
49                 StructField("score", DoubleType(), True)
50             ]),
51             True
52         )
53     ])
54
55     pedidos_data = [
56         (
57             "id-1",
58             "PRODUTO A",
59             100.0,
60             2,

```

tests/test_business_logic.py - linhas 61-80

```

61         "2025-03-01T00:00:00.000000",
62         "SP",
63         "cliente-1"
64     ),
65     (
66         "id-2",
67         "PRODUTO B",
68         200.0,
69         1,
70         "2024-03-01T00:00:00.000000",
71         "RJ",
72         "cliente-2"
73     ),
74 ]
75
76     pagamentos_data = [
77         (
78             "id-1",
79             "PIX",
80             200.0,

```

tests/test_business_logic.py - linhas 81-100

```

81         False,
82         (False, 0.1)
83     ),
84     (
85         "id-2",
86         "BOLETO",
87         200.0,
88         False,
89         (False, 0.1)
90     ),
91 ]
92
93 df_pedidos = spark.createDataFrame(
94     pedidos_data,
95     schema=pedidos_schema
96 )
97
98 df_pagamentos = spark.createDataFrame(
99     pagamentos_data,
100    schema=pagamentos_schema

```

tests/test_business_logic.py - linhas 101-117

```

101     )
102
103     logic = BusinessLogic()
104
105     resultado = logic.process(
106         df_pedidos,
107         df_pagamentos
108     )
109
110     assert resultado.count() == 1
111
112     linha = resultado.first()
113
114     assert linha["ID_PEDIDO"] == "id-1"
115     assert linha["UF"] == "SP"
116     assert linha["FORMA_PAGAMENTO"] == "PIX"
117     assert linha["VALOR_TOTAL_PEDIDO"] == 200.0

```

4. Execução e testes

Comandos utilizados para configurar, executar e testar o projeto:

- `pip install -r requirements.txt`
- `python -m src.main` – executa o pipeline e gera o relatório Parquet em `output/relatorio/`
- `pytest tests/` – executa o teste unitário da classe `BusinessLogic`

Teste executado com sucesso (`pytest tests/`) no ambiente Spark utilizado na disciplina.

Repositório público no GitHub (código completo, README com instruções de instalação e execução):

github.com/Abethiol/fiap-pyspark